BaitFL: Lightweight Trap-Based Free-Rider Detection and Mitigation in Federated Learning

Aashnan Rahman
Department of Computer Science and Engineering
Islamic University of Technology

***Abstract*—Free riders obtain the shared federated model without performing local training, while fabricated responses can resemble genuine updates. BaitFL audits this behavior through undisclosed model challenges, update norms and layer-level profiles. Geometric tests and repeated fresh evidence govern removal, while bounded review limits exclusion when the evidence remains inconclusive. The server requires neither an auxiliary detector nor client-reported loss or attack identities. Experiments across three random seeds cover MNIST and CIFAR-10, IID and non-IID partitions, four non-adaptive fabrication strategies, and 30% and 40% free riders. Across the 96 runs, all evaluated free riders are removed, yielding 100.00% recall and 99.91% pooled precision, with 3 honest removals. Separately instrumented 100-round executions place checking at 0.58% of MNIST computation time and 0.89% on CIFAR-10. The measured response-checking cost is small for the evaluated population of 100 clients. Honest-response overlap and withheld training contributions remain costs of the method, which requires visibility of individual client responses.**

***Index Terms*—federated learning, free-rider detection, hidden challenges, robust statistics, non-IID data, client auditing**

I. Introduction

Federated learning (FL) allows multiple clients to train a shared model without handing over their local data. In each communication round, the server distributes the current global model, the clients update it using their own data, and the server aggregates the returned updates into a revised model [1]. Since raw data stays with its owner, participants can collaborate without first creating a central repository of their records. Keeping data local does not remove every privacy risk, as shared updates can still reveal information about training data [2]–[4]. Exchanging updates instead of datasets also avoids moving large volumes of local data, although repeated model exchange remains a significant communication cost in FL [1]. Effective collaboration depends on clients performing useful local training, even though the server cannot directly observe that work.

Like other distributed systems, FL is open to misconduct. Label-flipping and model-poisoning attacks, for example, alter local data or submitted updates in order to degrade the global model or introduce targeted behavior [2], [5]. Free riding is a quieter problem. The participant's primary aim is to obtain the shared model without paying the cost of local training [6], [7]. A client may lack useful data or computational resources, or may simply choose to conserve them. Instead of training, it can return an unchanged model, fabricate a noisy response, or build an update from models received in earlier rounds. Such responses may resemble what the server expects, so they need not visibly disrupt learning and can evade checks aimed at overtly harmful updates. Free riding shifts the training burden to honest participants. When a client withholds useful data and computation, the federation also loses a contribution that could have improved the shared model. Figure 1 illustrates this behavior: the free rider receives the shared model alongside the training clients but returns a fabricated response instead of performing local optimization.

Detecting this omission is difficult because unusual updates are not limited to free riders. Honest clients differ in sample sizes, class distributions, and local learning trajectories, particularly under non-IID data [2]. A small update can therefore reflect legitimate training, while a fabricated one can look like an ordinary contribution. The server has to tell these cases apart without observing local optimization. Existing approaches rely on learned anomaly models, parameter audits, or evidence obtained through inference attacks [6], [8], [9]. This raises a practical question: can lightweight server-side checks detect free riding without training an auxiliary detector? There is also a cost to being wrong. Withholding an honest client's updates can reduce its contribution to the shared model, even if it is never permanently removed. A detector must therefore consider both the evidence needed for removal and the duration of temporary exclusion.

To address this problem, we propose *BaitFL*, a lightweight trap-based method for detecting and mitigating free riders in federated learning. Across the evaluated image-classification tasks, it removes all tested free riders with 99.91% pooled precision, while checking accounts for less than 1% of computation in the instrumented executions.

The main contributions of this work are as follows:

- **Lightweight server-side detection.** We develop a challenge-based detector using update norms and tensor profiles, without training an auxiliary detection model or accessing clients' private training data.
- **Confirmation using repeated evidence.** We design a decision mechanism that separates initial suspicion from permanent removal and requires fresh evidence of the same component across two challenges and distinct audit cycles for statistical confirmation.
- **Bounded mitigation and rehabilitation.** We retain strong evidence across ambiguous cycles while allowing unconfirmed quarantine to expire and clearing statistical memory after repeated clean observations.

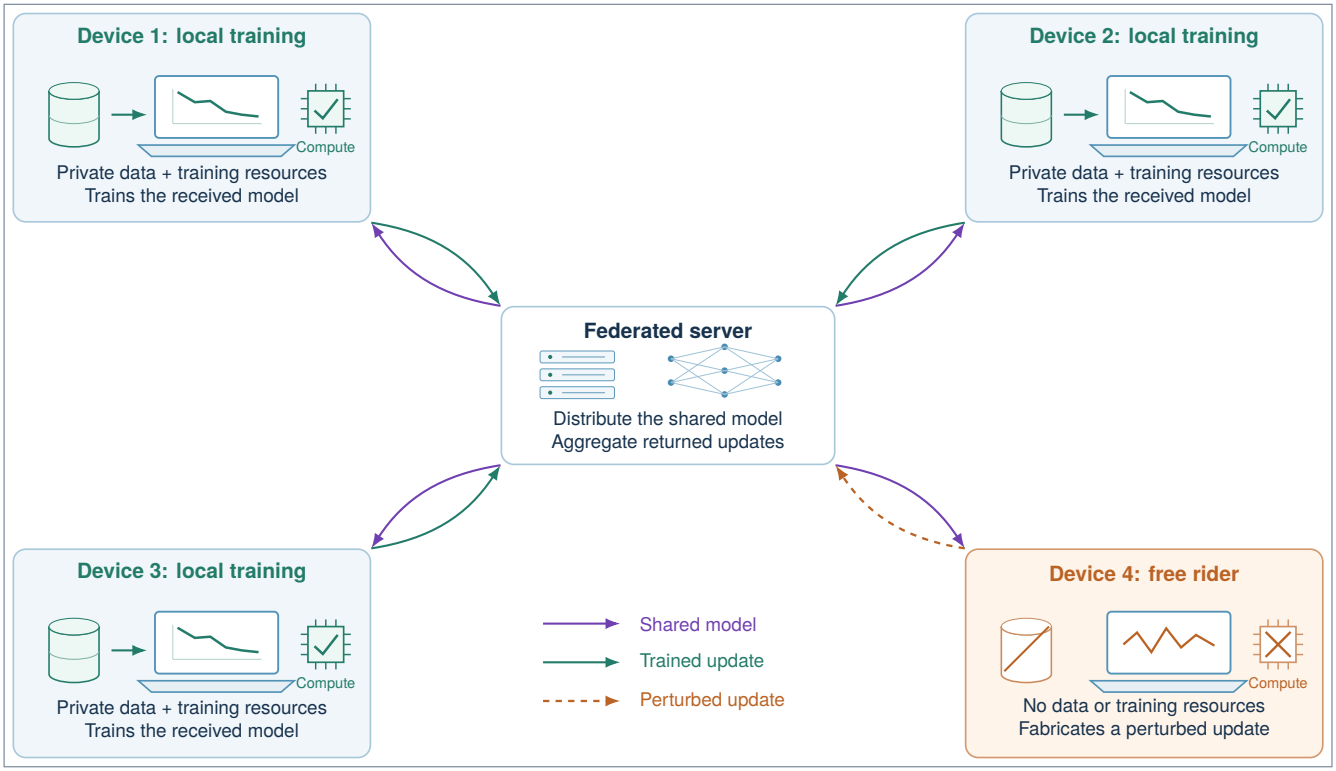


Fig. 1. Free riding in federated learning.

- **Evaluation across four free-rider behaviors.** We study unchanged-model responses, random fabrication, history averaging, and trajectory imitation on MNIST and CIFAR-10 under IID and non-IID partitions, with 30% and 40% free riders.

Sections II and III review related work and define the threat model. Section IV presents the auditing procedure, followed by results in Section V and complexity analysis in Section VI. Section VII discusses security, limitations and future work before the conclusion.

II. Literature Review

A. Free-Rider Attacks and Evasion

Lin *et al.* examined how clients could fabricate updates without local training data, using random weights or changes in the global model [6]. Their experiments established that receiving and returning a model does not demonstrate participation in training. They also showed why a defense that catches one fabrication strategy may fail when the attacker changes how the response is constructed.

Fraboni *et al.* provided a theoretical and experimental account of free riding under iterative aggregation [7]. Stochastic disguises allowed non-contributing clients to approach the model obtained by honest participants. Zhu *et al.* studied the evolution of local and global models through an Ornstein–Uhlenbeck process and developed attacks that account for both convergence and concealment [10]. For detection, the difficulty is clear: a fabricated response can follow the same broad trajectory as a trained one. Similarity to ordinary updates is therefore weak evidence of genuine work, just as an unusual update need not establish free riding.

B. Learned Detection and Contribution Auditing

STD-DAGMM, proposed by Lin *et al.*, combines an autoencoder and mixture model with the standard deviation of submitted updates [6], [11]. The added statistic improved separation where DAGMM alone struggled. That improvement comes with an auxiliary model to fit and retain, adding computation and memory beyond the update statistics themselves. The cost depends on the architecture and fitting schedule. Our measurements are reported separately in Section V-H. Lin *et al.* considered differing local data distributions, although their experiments used MNIST and a shallow fully connected model. An anomaly score also remains an indirect signal: it identifies a response that differs from its peers, without observing whether local training actually occurred.

FRAD extends learned detection with contribution and reputation. Wang *et al.* model contribution through computing resources, communication cost and data quality, then use PageRank-based reputation to guide participation [12]. Historical information gives the detector more context than a single update. It also creates a practical requirement for reliable estimates of those quantities, while retaining DAGMM fitting. A

low contribution ranking, on its own, does not explain whether a client withheld training or simply had less to contribute.

PASS takes a different route. Clients evaluate other participants' updates on their own private datasets and report accuracy-based contribution scores [8]. The method addresses both anonymous and selfish free riders, including evaluated cases with an attacker majority. Auditing stays at the clients, so it does not require the server to inspect their raw data. Participating devices must nevertheless evaluate additional models, receive other clients' updates and report audit scores. Sharing individual updates with peers also extends their exposure beyond the server. PASS addresses this risk through Gaussian perturbation and parameter pruning, making privacy protection and audit utility part of the same design choice. An accuracy-based audit measures usefulness on the evaluator's data. Under heterogeneous data, poor utility alone is not conclusive evidence that the sender skipped training.

C. Canary-Based Evidence and Privacy Tradeoffs

Recasens *et al.* introduced FRIDA, which uses membership and property inference to look for evidence of client training [9]. Its membership-based variants ask clients to train on a small labeled canary set supplied by the server. The server then tests returned models using canary losses or gradient similarity. This creates a deliberate training signal, but clients must perform extra canary training and the server must hold suitable examples and evaluate responses. The variants also behave differently under evasion. In the authors' canary-only training experiment, the cosine-based test was evaded, whereas the loss-based test retained separation.

FRIDA's property-based variants avoid client-side canary training by inferring characteristics such as label distributions from updates [9]. They move work to the server and use information about local datasets as detection evidence. Raw records need not leave the client, yet inferred properties can still be sensitive. The authors examine privacy protections, including local differential privacy. The tradeoff concerns what can be inferred from an update, not direct access to the underlying records.

There is a related constraint on all methods that inspect individual responses. Secure aggregation, developed by Bonawitz *et al.*, conceals those responses while revealing their aggregate [13]. A detector that needs attributable updates cannot operate unchanged behind that protection. BaitFL has this limitation too. Keeping raw data local does not, by itself, make the submitted model private.

D. Robust Aggregation and Heterogeneous Training

Robust aggregation addresses corrupted contributions rather than directly verifying participation. Krum selects updates through pairwise distances [14]. Coordinatewise median and trimmed mean resist contaminated coordinates [15], while RFA uses a geometric-median oracle that can build on secure averaging [16]. These methods can limit harmful influence without establishing whether a participant trained. A free rider whose response lies inside the accepted update population may still benefit from the shared model.

FLTrust instead trains a server model on a small trusted root dataset and uses its update to assess client directions [17]. That trusted-data assumption differs from BaitFL's population-based comparison. Neither design establishes that a response with a plausible direction necessarily came from local optimization.

Legitimate heterogeneity also matters. FedProx limits local drift with a proximal objective, SCAFFOLD uses control variates, and FedNova addresses objective inconsistency from unequal local work [18]–[20]. Such methods improve the training process. They do not, by themselves, identify clients that skipped it. Combining auditing with a different optimizer would require reevaluating both honest-response variation and fabricated responses.

E. Verifiable Training and Contribution Valuation

Proof-of-learning approaches retain evidence from the optimization trajectory. Jia *et al.* use training records and partial replay to verify claimed work [21]. VeriML uses cryptographic proofs for selected committed training iterations in outsourced machine learning [22]. These approaches offer a different form of assurance, with requirements for producing, retaining and checking training evidence. BaitFL instead assesses returned models and does not provide a cryptographic proof of computation.

Contribution valuation asks a related but distinct question: how much a participant's data or update improves a chosen utility. Data Shapley and efficient Shapley approximations formalize this allocation problem [23], [24]. GTG-Shapley reduces the expense through gradient-based model reconstruction, guided sampling and truncation [25]. Marginal utility is not evidence of training effort. Honest clients can contribute little on a particular evaluation distribution, and a fabricated response can still receive a nonzero valuation.

These studies provide several forms of evidence, each with a cost: auxiliary fitting, peer evaluation, proof verification, model evaluation or inference about private data. The question pursued here is whether useful checks can be built into ordinary model exchanges without adding an auxiliary detector or a separate client-side audit task. BaitFL examines responses to hidden model challenges and requires repeated evidence before removal. Its challenge responses are withheld from aggregation, however, so the cost of those checks and their effect on honest participation remain part of the evaluation.

The comparisons serve different purposes. We measure BaitFL's detection and utility under the stated attacks, compare utility with no-defense aggregation, and profile a STD-DAGMM implementation for timing. Table I compares published computation and information requirements. It is not an empirical ranking of detection accuracy, runtime or privacy across all the listed methods.

III. Threat Model

A. Participants and Server Knowledge

A trusted server coordinates N clients with persistent identities and reliable communication. Every active client responds in each round. The population is fixed, with no client sampling, late arrivals or dropouts. Removal ends participation. Quarantined clients still return responses, but those responses are excluded from aggregation, as are all responses to traps. Honest clients train the model they receive with the prescribed optimizer. Their datasets may differ in size and class distribution, covering both IID and non-IID partitions.

The server may send different models to different clients without disclosing which are traps. For client i in round t, it knows the sent model, $w_{t,i}^{\text{send}}$, and the returned model, $w_{t,i}^{\text{ret}}$. These give the update and its norm,

$$\Delta_{t,i} = w_{t,i}^{\text{ret}} - w_{t,i}^{\text{send}}, \qquad r_{t,i} = \|\Delta_{t,i}\|_2. \tag{1}$$

Detection uses these responses and the server's audit history. The server has no access to private training records and no prior knowledge of which clients are free riders or how they construct their responses. Individual responses are visible to it, so the setting does not use secure aggregation.

B. Free-Rider Objective and Scope

Free riders seek the shared model while avoiding local training costs. Some lack suitable data or training resources. Others have both but choose to withhold them. They can still receive and store models and carry out the relatively simple calculations needed to fabricate a response. Introducing a targeted error or backdoor is outside their objective here.

Our experiments consider persistent free riding: these clients perform no local optimization at any stage. Partial training and behavior that alternates between training and free riding fall outside this scope. Each attacker relies only on its own received-model history and local randomness. There is no collusion, response sharing or identity change. The evaluated strategies do not attempt to recognize traps or adapt to evade audits. Section VII-C discusses those possibilities.

We assume an honest majority because screening uses peer statistics and seeks a separated minority of suspicious responses. The experiments use 30% and 40% free riders, with one fixed fabrication strategy shared by the free riders in each run.

C. Evaluated Free-Rider Behaviors

Let $\widehat{w}_{t,i}$ denote the model returned by a free rider. All four strategies below skip local optimization.

FR1: current-model sendback.

Returning the received model unchanged gives a zero update, whether the model is ordinary or a trap:

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}}. \tag{2}$$

FR2: bounded random fabrication.

Independent uniform noise is added to each floating point entry of the received model state:

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}} + \epsilon_{t,i}, \quad (\epsilon_{t,i})_j \sim \mathcal{U}(-10^{-3}, 10^{-3}). \tag{3}$$

FR3: received-history averaging.

With a history length H, the response averages the most recent $\min(H, t)$ received model states, stored in $\mathcal{H}_{t,i}$:

$$\widehat{w}_{t,i} = \frac{1}{|\mathcal{H}_{t,i}|} \sum_{w \in \mathcal{H}_{t,i}} w. \tag{4}$$

This history includes the current model and any earlier traps. The client therefore follows recent model evolution without training on local data.

FR4: received-trajectory imitation.

Changes between consecutive received models provide an estimated update. Let $\overline{d}_{t,i}$ average up to the five most recent differences:

$$d_{s,i} = w_{s,i}^{\text{send}} - w_{s-1,i}^{\text{send}}, \tag{5}$$

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}} + \overline{d}_{t,i} + \eta_{t,i}. \tag{6}$$

For a standard Gaussian vector $z_{t,i}$, the noise is scaled as

$$\eta_{t,i} = 0.1 \, \frac{\max(\|\overline{d}_{t,i}\|_2, \epsilon_0)}{\max(\|z_{t,i}\|_2, \epsilon_0)} \, z_{t,i}, \qquad \epsilon_0 = 10^{-6}. \tag{7}$$

The vectors cover all floating model entries. On the first response, $\overline{d}_{1,i} = 0$ and $\eta_{1,i}$ instead has independent Gaussian entries with standard deviation 10^{-4}. Traps enter the same history as ordinary models and affect the estimated trajectory.

IV. Proposed Methodology

BaitFL compares clients through hidden model challenges, then separates initial suspicion from the evidence needed for removal. Figure 2 shows the four phases. Warmup establishes an ordinary training trajectory (Algorithm 1). Paired coverage collects comparable responses (Algorithm 2), and confirmation updates the client records (Algorithm 3). Outside paired cycles, ordinary training continues with sparse audits (Algorithm 4).

The server first runs warmup and initial paired coverage. Thereafter, it repeats paired cycles while any client has an active review budget or rehabilitation probation. Otherwise, it executes ordinary rounds and any scheduled sparse audit until the training horizon T is reached. These transitions are the feedback arrows in Figure 2. The records $\mathcal{S}$ retain client status,

TABLE I
REQUIREMENTS OF FREE-RIDER AUDITING APPROACHES.

Approach	Extra learning	Additional client work	Server work	Information and exposure
STD-DAGMM [6]	Autoencoder and mixture fitting	No separate peer audit	Feature construction, fitting and density scoring	Individual response features are inspected.
FRAD [12]	DAGMM fitting	Contribution-related inputs	Learned detection and reputation updates	Contribution/resource information supplements response evidence.
PASS [8]	No separate anomaly learner	Evaluate other clients' models and report scores	Coordinate parameter audits and participation	Peer models reach auditing clients, with perturbation/pruning protections.
FRIDA canary variants [9]	Client canary training	Train on server-provided examples	Canary loss or gradient tests	Server holds labeled canaries and inspects individual responses.
FRIDA property variants [9]	Attack-dependent inference	No canary training	Infer local-data characteristics	Inferred dataset properties form the detection evidence.
BaitFL	No auxiliary learning	Train one ordinary or challenge model	Norms, profiles, statistical splits and confirmation	Individual responses remain visible. Challenge responses are withheld from aggregation.

Privacy is compared through required information and exposure, not through measured leakage rates. BaitFL does not provide differential privacy or directly support standard secure aggregation.

quarantine, review budget, component counts, clean progress and cooldown. Counts start at zero and the quiet clock is initially unset, $\sigma = \perp$. The study uses warmup length $W = 10$, paired-cycle length $\ell_{\mathrm{cyc}} = 10$ and sparse-audit interval $I = 10$.

The challenge changes a client's starting point without changing the requested training task. Honest optimization responds to that starting point through the local data. An unchanged response, arbitrary noise or recycled model history need not respond in the same way. Comparing clients that received the same challenge can expose differences in the size and layer distribution of their updates. Two independent challenges test whether the difference recurs. This is behavioral evidence rather than proof of training, since fabricated and genuine responses may still overlap.

A. Challenges, Measurements, and Aggregation

Each paired cycle starts from a frozen copy $w^{(c)}$ of the current global model. For floating tensor k, the server creates traps A and B as

$$\tau_{a,k} = w_k^{(c)} + \gamma \max\{\mathrm{sd}(w_k^{(c)}), \epsilon_\tau\}\xi_{a,k}, \quad a \in \{A, B\}. \quad (8)$$

Here $\xi_{a,k} \sim \mathcal{N}(0, I)$ uses independent draws, and sd is the tensor's population standard deviation. The study sets $\gamma = 0.01$ and $\epsilon_\tau = 10^{-4}$. Floating buffers, including BatchNorm statistics, are perturbed along with weights. Integer counters remain unchanged. The two traps stay fixed within the cycle, even as ordinary responses update the global model.

For a response to trap a, write $\Delta_{i,k}^a$ for the returned tensor minus the tensor sent to client i. Its norm r_i^a measures total update magnitude. The normalized tensor profile is

$$p_{i,k}^a = \frac{\|\Delta_{i,k}^a\|_2}{\sum_j \|\Delta_{i,j}^a\|_2}. \quad (9)$$

The profile is zero when its denominator is below 10^{-6}. Otherwise, it records where the update magnitude lies across floating tensors.

For each trap, client i is compared with all other active clients having a selected response to that same trap. Quarantined and unresolved clients are not removed from this reference. Let m_i^a and d_i^a be the median and median absolute deviation (MAD) of the peer norms. Then

$$z_i^{n,a} = \frac{r_i^a - m_i^a}{d_i^a / \Phi^{-1}(0.75)}, \quad (10)$$

where Φ is the standard normal cumulative distribution function. The factor normalizes the MAD. It does not assume that client updates are normally distributed.

The profile reference $\widetilde{p}_{-i}^a$ is the coordinatewise median of the peer profiles. The implementation uses the lower middle value when a tensor coordinate has an even number of peer observations. Define the client's profile distance as $\rho_i^a = \|p_i^a - \widetilde{p}_{-i}^a\|_2$. Peer distances are computed against that same reference. Their median μ_i^a and MAD δ_i^a give

$$z_i^{p,a} = \frac{\rho_i^a - \mu_i^a}{\delta_i^a / \Phi^{-1}(0.75)}. \quad (11)$$

A missing comparison or zero MAD leaves the corresponding score undefined.

Send-back is checked separately. It means an exactly zero floating-point update norm, or a returned model whose fingerprint matches the immediately previous model sent to that client. The latter comparison covers every tensor in the model state. The server retains one previous-sent fingerprint per client, not a history of full returned models.

Aggregation runs alongside these checks. Let $\mathcal{A}_t$ contain clients that received the ordinary model and were not quarantined at the start of round t, excluding any newly quarantined or removed at its end. Then

$$w_{t+1} = w_t + \frac{1}{|\mathcal{A}_t|} \sum_{i \in \mathcal{A}_t} \Delta_{t,i}. \quad (12)$$

The update applies to floating tensors with equal client weights. This gives each eligible client equal influence, unlike sample-size weighting in the usual FedAvg formulation [1]. The server

TABLE II
PRINCIPAL NOTATION AND FIXED PROTOCOL SETTINGS.

Symbol	Meaning	Study setting
N, T	Initial client population and training horizon	100 clients, 100 rounds
w_t, τ_A, τ_B	Global model and the two shared challenges	Updated globally, fixed per cycle
Δ_i^a, r_i^a, p_i^a	Response change, its norm and its tensor profile	Measured from returned models
$z_i^{n,a}, z_i^{p,a}$	Leave-one-out norm and profile scores	Population-derived
W, ℓ_{cyc}, I	Warmup, paired-cycle length and sparse-audit interval	10, 10 and 10 rounds
γ, ϵ_τ	Relative challenge strength and tensor-scale floor	0.01, 10^{-4}
B, q_{boot}	Bootstrap replicates and nominal split-interval level	1,000 and 99%
$q_{\text{confirm}}, q_{\text{clear}}$	Repeated evidence required and clean cycles to clear it	Two complete cycles each
$R_{\text{review}}, C_{\text{audit}}$	Review budget and release cooldown	Two cycles, two sparse audits
P_{ref}, G	Extra clean-cycle probation and maximum groups	One clear cycle, ten groups
P, L	Floating model coordinates and floating tensors	Architecture-dependent

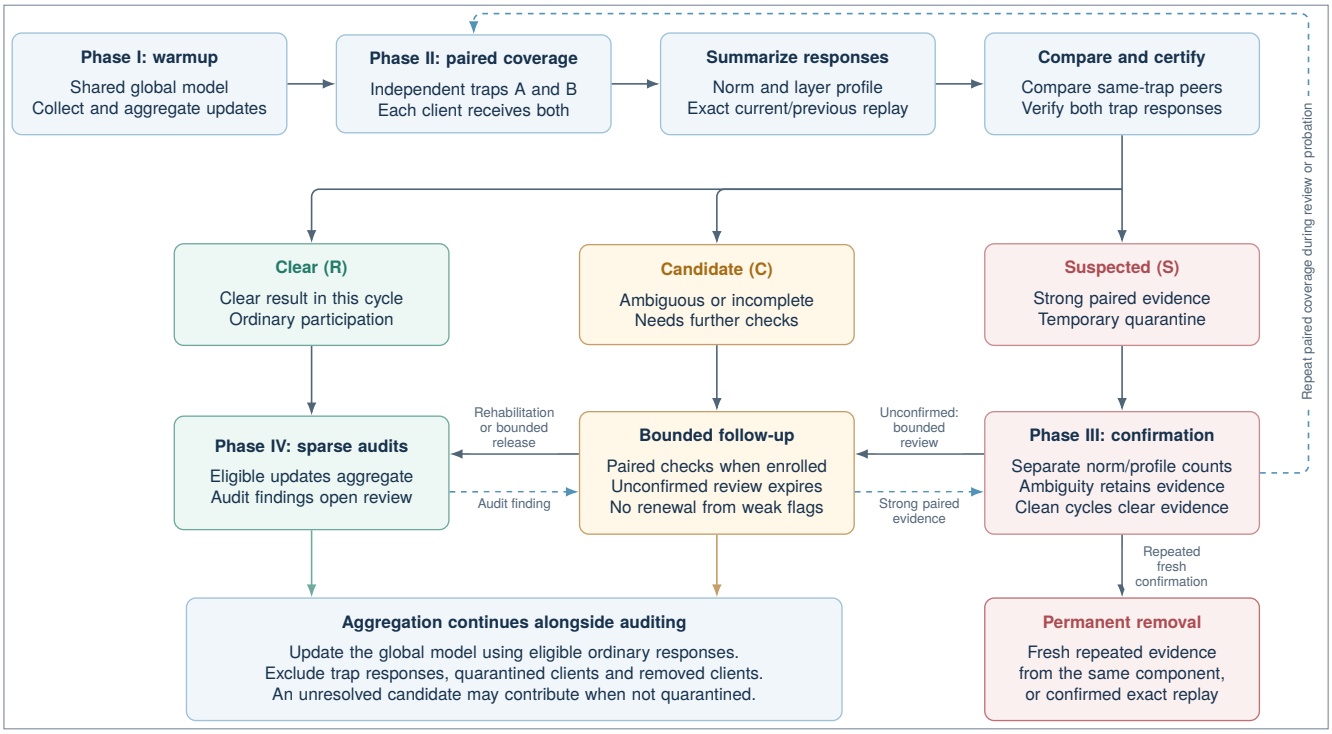


Fig. 2. BaitFL methodology.

does not need client sample counts. Both the detector study and its no-defense comparison use this same weighting, so their utility differences are evaluated under a common aggregation objective. Non-floating entries retain their server values. An empty $\mathcal{A}_t$ leaves the model unchanged. Training on a trap replaces ordinary local training in that round. The trap response never enters aggregation.

B. Phase I: Warmup

The warmup box in Figure 2 corresponds to Algorithm 1. All clients receive the ordinary global model and return responses for aggregation (lines 3–6). The server records update summaries and maintains the previous-sent fingerprints, but makes no removal decision. Evidence counters start at zero.

Algorithm 1 Phase I: ordinary warmup

1: **procedure** WARMUP($w, \mathcal{S}$)
2: **for** $t = 1, \ldots, W$ **do**
3: Send w to all active clients
4: Collect responses, summaries and send-back checks
5: $\mathcal{A}_t \leftarrow$ all active clients
6: $w \leftarrow$ AGGREGATE($w, \mathcal{A}_t$)
7: **end for**
8: **return** $(w, W, \mathcal{S})$
9: **end procedure**

Warmup ends without selecting trusted clients. The next phase starts with the entire active population and zero confirmation counts.

C. Phase II: Paired Coverage and Screening

The paired-challenge, response-summary and comparison boxes in Figure 2 form this phase. Initially, clients are shuffled into at most ten groups. Pairs of groups receive one trap, then the opposite trap in the second half of the cycle. Every client receives both. With 100 clients, about 20% receive a trap in each initial-coverage round. The rest receive the current global model.

Later cycles reuse Algorithm 2, with different scheduling for clients under review. For each client and trap, one response position is chosen uniformly from its scheduled positions before responses are observed (line 10). Only those preselected summaries enter the cycle decision. Missing paired coverage cannot authorize removal. After a complete cycle, scoring and confirmation precede its final aggregation (lines 20–22).

Algorithm 2 Phase II: collect and assess paired responses

1: **procedure** PAIREDCYCLE($w, t, \mathcal{S}, c, \sigma,$ initial)
2: Create fixed independent traps A, B from a copy of w
3: **if** initial **then**
4: Shuffle groups and schedule one response to each trap
5: **else**
6: Group active-review and remaining clients separately
7: Alternate traps for reviewed clients throughout the cycle
8: Schedule each remaining client once on each trap
9: **end if**
10: Preselect one position per client and trap
11: Initialize selected summaries O
12: $h \leftarrow \min(\ell_{\text{cyc}}, T - t)$
13: **for** $s = 1, \ldots, h$ **do**
14: Advance t and read trap recipients $\mathcal{T}_t$
15: Send the trap to $\mathcal{T}_t$ and w to others
16: Collect responses and retain preselected summaries
17: $\mathcal{A}_t \leftarrow$ ordinary, non-quarantined recipients at round start
18: **if** $s = \ell_{\text{cyc}}$ **then**
19: Require complete A/B coverage of active clients
20: $E \leftarrow$ SCREENANDCERTIFY(O)
21: $\mathcal{S} \leftarrow$ CONFIRM($\mathcal{S}, E, c$)
22: Exclude new removals and quarantines from $\mathcal{A}_t$
23: **end if**
24: $w \leftarrow$ AGGREGATE($w, \mathcal{A}_t$)
25: **end for**
26: **if** the cycle completed **then**
27: Advance c and clear the completed schedule
28: $\sigma \leftarrow t$ if no review/probation remains, else $\perp$
29: **end if**
30: **return** $(w, t, \mathcal{S}, c, \sigma)$
31: **end procedure**

1) Screening: For each client, form $D_i^A = z_i^{p,A} - z_i^{n,A}$, $D_i^B = z_i^{p,B} - z_i^{n,B}$ and their average. Each view uses $\lceil \sqrt{n_v} \rceil$ equally spaced histogram bins, where n_v counts clients with valid paired scores. Otsu's criterion selects a split [26]. The lower side of the average-view split must contain a majority of all active clients, not merely a majority of the valid-score subset.

The server resamples clients jointly across the three views in 1,000 bootstrap replicates, obtaining a 99% interval for each split boundary [27]. A client above every upper boundary is nominated as suspicious. One below every lower boundary is clear. Other outcomes remain candidates. These intervals describe split stability, not a calibrated false-removal probability.

If the population split is unavailable or unstable, negative norm-z scores on both traps nominate suspicion. Here zero denotes the measured peer median. Neither score negative gives a clear result. One negative score or an undefined comparison leaves a candidate. Paired send-back evidence is handled independently of this statistical fallback.

2) Geometric evidence: A statistical nomination must pass a separate separation test. Let $\mathcal{C}$ be the nominated cohort and $\mathcal{P}$ the remaining active clients. With n active clients, statistical certification requires

$$\left\lceil \frac{n}{\lceil \sqrt{n} \rceil} \right\rceil \leq |\mathcal{C}| < |\mathcal{P}|. \tag{13}$$

The minimum support grows approximately with $\sqrt{n}$, while the strict minority condition reflects the assumed honest majority. This is a population-based support rule, not an estimate of the attacker share. For a given signal, write $Q_q(\mathcal{C})$ for its empirical quantile and $I_C = Q_{.75}(\mathcal{C}) - Q_{.25}(\mathcal{C})$ for its interquartile range (IQR). Low raw update norms require both

$$\begin{aligned} Q_{.25}(\mathcal{P}) - Q_{.75}(\mathcal{C}) &> I_C, \\ \min(\mathcal{P}) - Q_{.75}(\mathcal{C}) &> I_C. \end{aligned} \tag{14}$$

The individual norm must also lie below the peer median.

For high raw profile distances, let $I_P = Q_{.75}(\mathcal{P}) - Q_{.25}(\mathcal{P})$ and $g = Q_{.25}(\mathcal{C}) - Q_{.75}(\mathcal{P})$. Require $g > I_C$ and $g > 0$, together with

$$Q_{.25}(\mathcal{C}) - \max(\mathcal{P}) > I_C \quad \text{or} \quad \frac{I_C}{|\mathcal{C}| - 1} < \frac{I_P}{|\mathcal{P}| - 1}. \tag{15}$$

The individual's profile distance must exceed the peer median. Constant populations, invalid measurements or insufficient cohort support do not produce a statistical certificate.

The norm test requires the suspicious cohort to remain separated from the nearest peer, making overlap sufficient to reject certification. The profile test permits either full separation or a more concentrated cohort after the interquartile gap is established. Dividing each IQR by population size minus one compares typical spacing rather than spread alone. This allowance limits the influence of extreme peer distances, but can also admit persistent honest profile outliers. It is a geometric heuristic, not a calibrated false-positive test.

The same component must certify a client on both traps. The diagonal score nominates clients but cannot itself certify removal. Paired send-back supplies a separate evidence component without the statistical cohort requirement. The result is S for strong paired evidence, C for unresolved evidence, or R for a clear cycle result. A nomination that fails certification becomes C without automatic quarantine. These are cycle outcomes. A client's retained status may still reflect its recovery history.

D. Phase III: Confirmation and Bounded Review

The confirmation and evidence-memory boxes in Figure 2 represent Algorithm 3. Separate counts $M_{i,n}$ and $M_{i,p}$ retain

norm and profile certificates. Only a fresh paired certificate increments its own component, once per distinct complete cycle (line 11). The confirming cycles need not be consecutive. An ambiguous cycle preserves these counts, while two consecutive complete R cycles clear them. Norm and profile evidence never combine into one confirmation.

Removal requires a fresh certificate whose component count reaches two (line 14). Retained evidence alone cannot remove a client. Exact send-back follows a different rule: the paired condition must hold in two consecutive complete cycles. A missing or interrupted pair breaks that streak. Under the study's warmup and cycle lengths, the earliest removal is round 30.

Algorithm 3 Phase III: confirm evidence and bound review

```
1:  procedure CONFIRM(S, E, c)
2:      Require one complete decision per active client and a newer cycle
3:      for all active clients i do
4:          J_i ← statistical components certified on both traps
5:          Advance or reset the consecutive paired send-back streak
6:          if the cycle result is R then
7:              Advance the consecutive-R certificate-clean streak
8:              Clear M_{i,n}, M_{i,p} after two consecutive R cycles
9:          else
10:             Reset the certificate-clean streak
11:             Increment M_{i,j} once for each fresh j ∈ J_i
12:         end if
13:         if a fresh component reaches two, or send-back is confirmed then
14:             Permanently remove i
15:         else if strong paired evidence is present then
16:             Quarantine i and renew its review budget to two cycles
17:             Reset recovery, probation and cooldown
18:         else
19:             Decrease the review budget toward zero
20:             Apply consecutive-R rehabilitation and probation
21:             if unconfirmed review has expired then
22:                 Release i and set a two-audit cooldown
23:             end if
24:         end if
25:     end for
26:     return S
27: end procedure
```

Strong evidence starts quarantine and a two-cycle review budget (line 16). Further fresh strong evidence renews it. Without that renewal, the budget decreases after each completed cycle. Expiry releases unconfirmed clients even if their evidence memory is nonempty (line 22). An unresolved candidate may therefore aggregate on ordinary rounds when it is not quarantined.

Two consecutive clear results rehabilitate a suspected or candidate client. One additional clear cycle completes probation before sparse auditing resumes. Unconfirmed release instead starts a cooldown lasting two sparse audits. Fresh paired evidence can still start review during that cooldown. Weak flags and stored counts cannot prolong exclusion or keep dense probing active by themselves.

Clients with an active review budget and the remaining clients form separate pools. At most ten groups are allocated proportionally, using nearest rounding with ties down and bounds for the pool sizes. Reviewed clients receive alternating traps throughout the cycle. Each remaining client receives each trap once. This is the feedback to paired coverage in Figure 2: it reuses Algorithm 2 with fresh traps and newly preselected positions.

E. Phase IV: Ordinary Training and Sparse Audits

When review and probation are empty, the server records the start of quiet operation, σ. Algorithm 4 then handles each round. Ordinary responses continue to aggregate until a sparse audit is due. The sparse-audit box in Figure 2 shows this branch and its return to follow-up.

An audit sends one fresh trap to every active client. All responses are excluded, leaving the global model unchanged for that round. Single-view screening nominates clients, but only a strict norm/profile certificate or exact send-back can open review. An eligible finding starts a two-cycle review without immediate quarantine (lines 5–10). This brings the next round back to paired coverage. The audit cannot increment or clear statistical certificate memory, or authorize removal.

Algorithm 4 Phase IV: ordinary rounds and sparse audits

```
1:  procedure SPARSEROUND(w, t, S, σ)
2:      t ← t + 1
3:      if active clients remain, σ ≠ ⊥ and t − σ ≥ I then
4:          Send one fresh trap to all active clients
5:          Collect same-trap norm/profile scores and send-back checks
6:          F ← clients with a single-trap certificate or send-back
7:          Save the clients currently in cooldown
8:          Decrease all audit cooldowns toward zero
9:          for all i ∈ F outside the saved cooldown set do
10:             Open two cycles of review without quarantine
11:             Reset clean progress and the paired send-back streak
12:         end for
13:         σ ← t if F = ∅, otherwise ⊥
14:         Keep w unchanged
15:     else
16:         Send w to active clients and collect responses
17:         A_t ← eligible ordinary clients
18:         w ← AGGREGATE(w, A_t)
19:     end if
20:     return (w, t, S, σ)
21: end procedure
```

The cooldown check uses its value before decrementing, so a released client waits through two sparse audits before re-enrollment. Statistical memory survives this enrollment, while the consecutive send-back streak is reset. A clean audit restarts the quiet clock. The clock update in line 13 uses the findings before cooldown filtering. A cooldown-only finding can therefore leave the clock unset without opening review. The availability implications are discussed in Section VII.

F. Choice of Protocol Settings

Table II lists the settings shared by every reported configuration. Ten groups and a ten-round cycle provide a simple coverage schedule. Two challenges and two confirming cycles are the smallest repeated-evidence design used here. Review

and cooldown lengths bound exclusion and immediate re-enrollment, while the additional clear cycle extends monitoring before sparse auditing resumes. These are design tradeoffs, not statistically optimal choices.

Perturbations are scaled to each tensor so that a common relative strength can be used across layers. The scale floor handles tensors with negligible variance. Bootstrap replicates assess split stability, and the nominal confidence level controls the width of those split intervals. It does not supply a false-removal probability. The defaults were adopted during development and then held fixed across the reported grid. There is no separately held-out tuning study or sensitivity experiment, so the results do not establish an optimal parameter setting.

V. Results and Discussion

A. Experimental Setup

The study covers MNIST and CIFAR-10, IID and Dirichlet non-IID partitions, FR1–FR4, and free-rider proportions of 30% and 40% [28], [29]. Each configuration is evaluated across three random seeds, giving 96 detection runs in total. The main detection, honest-client impact and resource tables average the completed replications, with the optimizer and detector settings held fixed.

Every federation has 100 clients and runs for 100 communication rounds. IID data are shuffled into equal-size client partitions. Non-IID label proportions follow a Dirichlet distribution with $\alpha = 0.5$, with allocation repeated if any client receives no samples. Partitioning, attacker identities, initialization, local training and audit scheduling vary through separate random streams.

Honest clients perform three local SGD epochs, using learning rate 0.01, momentum 0.9 and batch size 32. The MNIST network has convolutional blocks with 16 and 32 channels and a 128-unit hidden classifier. CIFAR-10 uses 32, 64 and 128 channels with BatchNorm and a 256-unit hidden classifier. Images are normalized without augmentation. Execution uses the CPU, with six PyTorch threads. The evaluated FR3 history length is $H = 5$.

Two comparisons serve different purposes. Completed, seed-matched no-defense runs assess the effect of the whole BaitFL workflow. Separately instrumented BaitFL and STD-DAGMM executions measure timing stages over the full horizon. The latter are timing cases, separate from the multi-seed detection averages. STD-DAGMM is used for timing only. PASS, FRAD and FRIDA enter the requirements comparison, not a matched detection benchmark.

B. Evaluation Metrics

The evaluation asks whether BaitFL identifies free riders, preserves useful participation and adds little computational work. We assess these questions through detection performance, model accuracy, honest-client impact and resource use.

Detection performance. Precision measures the proportion of permanently removed clients that are free riders, while recall measures the proportion of free riders removed. We also report F_1 and the round at which removal of all free riders is complete. A candidate label or temporary quarantine does not count as detection. If any free rider remains at the training horizon, complete detection is recorded as not reached.

Model accuracy and honest participation. Global test accuracy measures the utility of the trained model. Comparisons with no-defense federated averaging report accuracy differences in percentage points. To assess the cost to honest participants, we record permanent removals, clients ever quarantined and unresolved candidates at completion. Client-round counts measure how long responses are withheld. Total withheld participation includes trap responses and quarantine, counting each affected client-round once.

Lightweightness. Mean round time, execution duration and peak process resident memory (RSS) describe the cost of the complete simulation. The round timer covers training, response processing, detection, aggregation and global evaluation, ending before log and checkpoint publication. Execution duration also includes setup and publication work, but excludes powered-off waiting. Separate instrumented runs divide the work into computation stages and distinguish these from elapsed round time, which also includes research I/O and other execution overhead. Timing means cover all 100 rounds, spreading periodic decision costs over the full training horizon. Percentage comparisons use the corresponding no-defense or STD-DAGMM execution as the reference. Network transmission is not measured in this local simulator.

Each configuration has three complete runs. We treat the run as the experimental unit and report accuracy variation across those runs. Pooled precision summarizes the observed client outcomes, not uncertainty from independent client trials. Clients in a federation share a training trajectory, so the pooled percentage is not a confidence bound.

C. Permanent Detection and Model Utility

Table III summarizes the multi-seed results. Detection rates combine the four attacks, while final model accuracy is reported separately for FR1–FR4 as mean $\pm$ sample standard deviation.

All free riders are removed in the evaluated replications, with pooled precision 99.91%, recall 100.00% and F_1 99.96%. Most executions complete detection at round 30. The slower cases extend to round 90, showing that successful detection need not occur at the first confirmation opportunity. Mean final test accuracy spans 98.83–99.02% on MNIST and 74.51–77.84% on CIFAR-10. Honest removals occur in two runs; per-run precision ranges from 95.24% to 100%. The pooled rate therefore summarizes errors concentrated in particular configurations rather than uniform behavior across runs.

FR1 and FR2 are consistently removed at the earliest confirmation round. FR1 returns an unchanged model and activates the exact-sendback path. FR2 avoids exact equality, but its fabricated noise remains distinct under the tested settings. The history-based attacks, FR3 and FR4, account for the delayed cases.

D. Honest-Client Impact

The principal errors occur among honest clients with non-IID data. Table IV reports 3 permanent honest removals and

TABLE III
DETECTION PERFORMANCE AND FINAL MODEL ACCURACY.

Dataset	Partition	FR (%)	Detection (%)		Final test accuracy (%)			
			Precision	Recall	FR1	FR2	FR3	FR4
MNIST	IID	30	100.00	100.00	99.01 ± 0.04	99.01 ± 0.04	99.02 ± 0.02	98.90 ± 0.07
		40	100.00	100.00	98.93 ± 0.06	98.94 ± 0.05	98.92 ± 0.07	98.83 ± 0.07
	Non-IID	30	100.00	100.00	98.92 ± 0.08	98.93 ± 0.06	98.93 ± 0.05	98.91 ± 0.05
		40	99.39	100.00	98.91 ± 0.07	98.91 ± 0.06	98.90 ± 0.05	98.88 ± 0.09
CIFAR-10	IID	30	100.00	100.00	77.84 ± 0.53	77.84 ± 0.31	77.76 ± 0.46	77.77 ± 0.37
		40	100.00	100.00	77.07 ± 0.55	77.13 ± 0.53	76.92 ± 0.52	76.97 ± 0.46
	Non-IID	30	100.00	100.00	75.94 ± 0.33	75.86 ± 0.65	75.77 ± 0.52	75.78 ± 0.36
		40	100.00	100.00	74.70 ± 1.22	74.88 ± 1.09	74.67 ± 0.64	74.51 ± 1.16

32 honest client-cases experiencing quarantine, totaling 620 quarantine client-rounds. No client remains quarantined at completion. Counts are averaged across runs. For instance, the 1.33 quarantined clients reported for MNIST non-IID FR3 at 30% is the mean number of honest clients placed under temporary review. The table includes FR1–FR4, including configurations with no honest quarantine or removal.

The decision records identify the immediate causes of the false positives. Two honest clients in MNIST non-IID FR3 receive profile-distance certificates in successive cycles and are removed at round 30. A third honest client, in FR4, receives norm certificates in separated cycles and is removed at round 90. None of these removals follows exact sendback. Non-IID training can produce legitimate updates whose size or distribution across layers differs persistently from the population. When these updates also satisfy the geometric gate, repeated confirmation can reinforce the same mistaken suspicion. Fresh traps do not remove the underlying similarity, because a client's local data remain unchanged. This is a limitation of inferring training behavior from response statistics.

Candidate labels can persist after the free riders have gone. Non-IID FR3 at 40% ends with mean honest-candidate counts of 37 on MNIST and 43 on CIFAR-10, respectively 61.7% and 71.7% of the 60 honest clients. That label alone does not prevent ordinary aggregation. Most withheld honest responses instead arise from the planned trap schedule. These forgone contributions are a utility cost even when the client is never removed.

E. Score Separation and Confirmation

Figures 3 and 4 show the round-20 response geometry for representative executions. Columns show FR1–FR4, and the rows distinguish proportion and partition. The plotted coordinates are $\operatorname{asinh}(\bar{z}_{\mathrm{norm}})$ and $\operatorname{asinh}(\bar{z}_{\mathrm{profile}})$, averaging a client's scores over its two selected challenges. The transform preserves ordering while compressing extreme tails. These are visual summaries, not the paired certificate used for removal.

FR1 and FR2 usually form distinct groups. FR3 shows why a statistical split alone cannot establish misconduct: responses assembled from received model history can overlap legitimate training responses. Non-IID data widen the variation among honest clients, making that overlap more consequential. The method therefore seeks fresh evidence of the same kind across both challenges and another complete cycle.

For example, the representative CIFAR-10 non-IID FR3 execution at 40% provides paired norm certificates at rounds 20 and 60. At 30%, the corresponding certificates arrive at rounds 40 and 80. Ambiguous cycles between them preserve earlier evidence without confirming removal. Evidence memory allows the later matching certificate to confirm removal after ambiguous intervening cycles. Colors in the plots identify evaluation labels only. They are never available to the server's decision rule.

TABLE IV
HONEST-CLIENT IMPACT.

Dataset	Partition	FR (%)	Attack	Mean honest clients per run			Excluded client-rounds		Responses withheld (%)
				Final candidates	Permanent removals [range]	Ever quarantined [range]	Quarantine	Ordinary responses	
MNIST	IID	30	FR1	1.00	0	0	0.00	0.00	11.00
			FR2	4.00	0	0	0.00	0.00	11.00
			FR3	6.33	0	0	0.00	0.00	11.00
			FR4	5.33	0	0	0.00	0.00	11.00
		40	FR1	0.67	0	0	0.00	0.00	11.00
			FR2	5.33	0	0	0.00	0.00	11.00
			FR3	4.33	0	0	0.00	0.00	11.00
			FR4	16.33	0	0	0.00	0.00	11.67
	Non-IID	30	FR1	5.00	0	0	0.00	0.00	11.00
			FR2	4.67	0	0	0.00	0.00	11.00
			FR3	41.67	0	1.33 [0–3]	26.67	1.00	12.32
			FR4	40.67	0	0	0.00	0.00	11.74
		40	FR1	15.00	0	0	0.00	0.00	13.18
			FR2	6.33	0	0	0.00	0.00	11.00
			FR3	37.00	0.67 [0–2]	3.33 [1–5]	60.00	2.67	12.86
			FR4	35.00	0.33 [0–1]	1.00 [0–2]	20.00	0.67	14.57
CIFAR-10	IID	30	FR1	2.33	0	0	0.00	0.00	11.00
			FR2	0.00	0	0	0.00	0.00	11.00
			FR3	2.33	0	0	0.00	0.00	11.00
			FR4	5.67	0	0	0.00	0.00	11.00
		40	FR1	0.33	0	0	0.00	0.00	11.00
			FR2	0.33	0	0	0.00	0.00	11.00
			FR3	4.00	0	0	0.00	0.00	11.00
			FR4	4.33	0	0	0.00	0.00	11.00
	Non-IID	30	FR1	17.00	0	0	0.00	0.00	11.64
			FR2	5.00	0	0	0.00	0.00	11.00
			FR3	42.67	0	0.67 [0–1]	13.33	0.00	15.02
			FR4	27.33	0	0.67 [0–2]	13.33	0.67	12.16
		40	FR1	6.00	0	0	0.00	0.00	11.00
			FR2	3.67	0	0	0.00	0.00	11.00
			FR3	43.00	0	3.67 [1–9]	73.33	2.00	17.08
			FR4	14.00	0	0	0.00	0.00	11.67

Entries are per-run means; brackets give observed minimum–maximum counts. A client-round is one honest client in one round. Withheld responses include traps and quarantine without double counting, as a percentage of active honest client-rounds. Ordinary exclusions count non-trap responses.

F. Execution Time and Memory

Table V reports multi-seed process measurements averaged over the attacks, with minimum and maximum execution values in brackets. Runs at 30% free riders include more honest training jobs than those at 40%, which helps explain their longer rounds. Process RSS includes training tensors, simulated attacker histories and server state. It should not be interpreted as detector-only memory.

The overall cost and the cost of the checking stages answer different questions. Table V describes the complete simulation. Section V-H measures selected operations, while Section VI analyzes their scaling.

G. Comparison with No-Defense Federated Averaging

The no-defense baseline keeps all clients throughout training and aggregates every response. It performs no traps, screening, quarantine or removal. Each completed baseline is matched to BaitFL using the same dataset, partition, attack, proportion and random seed. Configuration checks and identical warmup accuracies verify that the paired runs start from the same training conditions.

Table VI uses 63 completed matched runs: 16 on MNIST and 47 on CIFAR-10. Means are calculated across matched seeds within each attack, then across the four attacks with equal weight. A BaitFL execution without a completed baseline partner does not enter this comparison, even though it remains in Table III. Positive accuracy changes favor BaitFL. A positive time change means longer rounds. Accuracy differences are expressed in percentage points.

The available comparisons improve mean final accuracy by 0.10 percentage points on MNIST and 0.60 on CIFAR-10. The benefit varies across attacks and partitions. MNIST rounds become modestly shorter, while the current CIFAR-10 comparisons take somewhat longer than no defense. These are whole-workflow measurements of the full training procedure. Stage-level timing is examined separately below.

The accuracy gains describe the combined effect of detection and mitigation. Removing fabricated responses changes aggregation, but challenge rounds also withhold genuine updates.

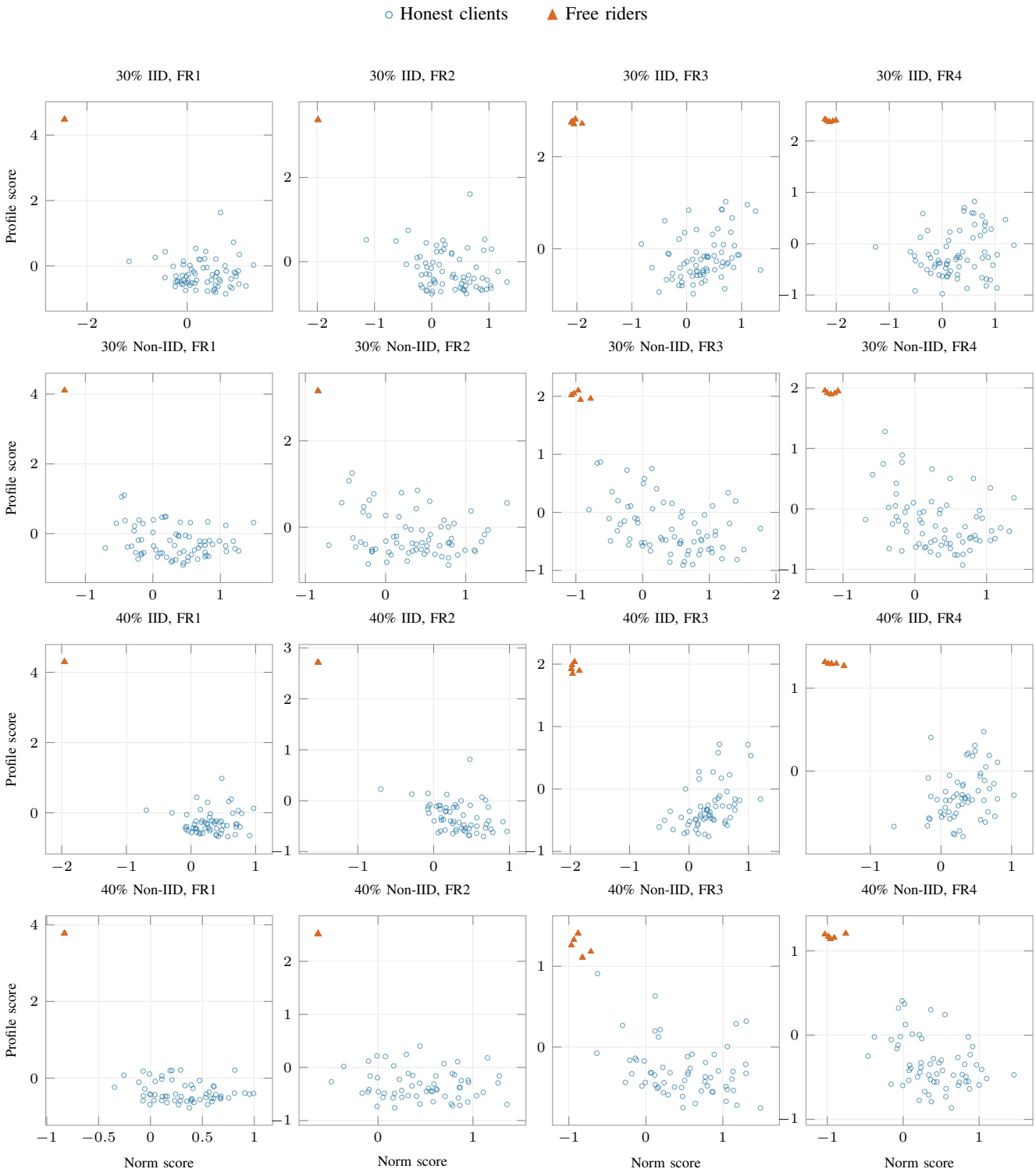


Fig. 3. MNIST client response scores.

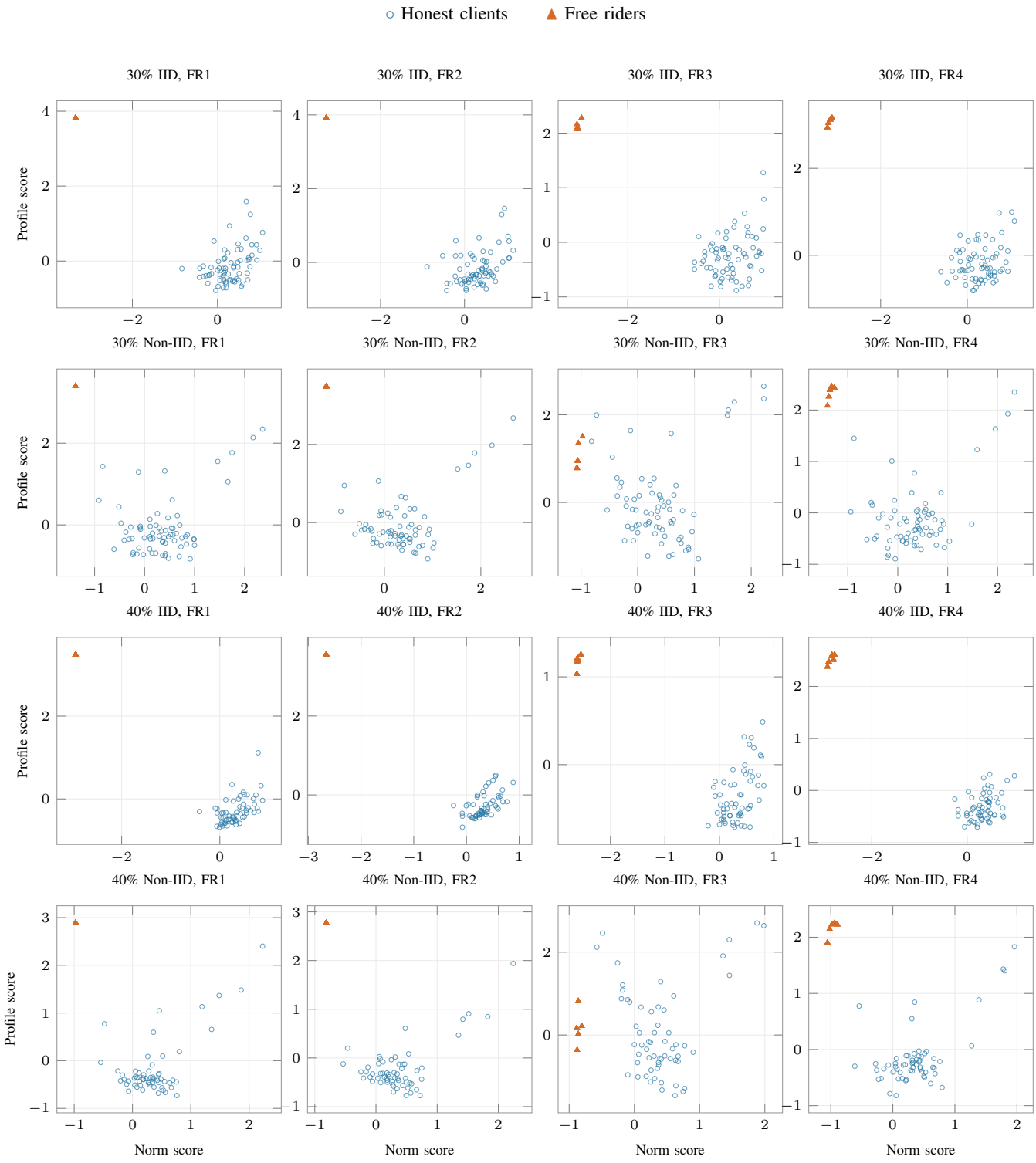


Fig. 4. CIFAR-10 client response scores.

TABLE V
EXECUTION TIME AND PROCESS MEMORY.

Dataset	Partition	FR (%)	Round time (s) mean [range]	Peak process RSS (GiB) mean [range]	Execution duration (min/run)
MNIST	IID	30	42.38 [39.78–48.57]	0.54 [0.29–0.78]	75.02
		40	37.05 [35.35–42.68]	0.60 [0.30–0.88]	66.71
	Non-IID	30	41.62 [40.57–43.43]	0.57 [0.30–0.78]	73.79
		40	36.08 [34.34–38.67]	0.62 [0.30–0.88]	65.27
CIFAR-10	IID	30	99.35 [88.18–119.07]	1.24 [0.70–1.76]	175.07
		40	85.35 [73.52–104.21]	1.38 [0.71–2.06]	154.65
	Non-IID	30	92.30 [82.85–108.38]	1.31 [0.73–1.79]	161.97
		40	77.03 [69.10–84.85]	1.49 [0.75–2.14]	139.40

TABLE VI
COMPARISON WITH NO-DEFENSE AGGREGATION.

Dataset	Partition	FR (%)	Final accuracy (%)			Mean round time (s)		
			No defense	BaitFL	Difference (percentage points)	No defense	BaitFL	Change (%)
MNIST	IID	30	98.92	98.96	+0.04	44.37	42.99	-3.10
		40	98.77	98.86	+0.09	37.04	36.43	-1.64
	Non-IID	30	98.90	98.99	+0.09	44.91	41.28	-8.08
		40	98.78	98.96	+0.18	35.97	34.57	-3.89
CIFAR-10	IID	30	77.35	77.76	+0.40	93.68	98.08	+4.69
		40	76.31	77.02	+0.71	81.28	85.35	+5.01
	Non-IID	30	75.35	75.84	+0.49	91.28	92.30	+1.12
		40	73.90	74.69	+0.79	74.52	77.03	+3.37

Final model utility therefore depends on both effects, rather than on removal recall alone.

Figure 5 shows model accuracy from rounds 20–100, averaging all three BaitFL replications and the completed no-defense runs. Baseline coverage is one replication per attack setting on MNIST and two to three on CIFAR-10. FR1–FR4 receive equal weight, and each dataset row shares an accuracy scale. The curves show how learning continues after initial challenge coverage. Coverage differs between methods, so the paired accuracy differences in Table VI remain the basis for the quantitative comparison.

H. Instrumented Timing and Lightweight Operation

Dedicated executions compare BaitFL with a reimplementation of STD-DAGMM on non-IID FR3 at 40%, using both datasets, 100 clients, 100 rounds, three local epochs and six CPU threads. STD-DAGMM freshly fits its auxiliary model for five epochs per round and retains every client. That fitting budget is a declared reproduction choice, not an author-specified training schedule or evidence of convergence [6], [11]. The instrumented BaitFL run matches its corresponding study accuracy trajectory in every round.

Table VII separates training from response preparation, challenge construction, decisions, aggregation and evaluation. BaitFL's recorded challenge, feature/preparation and decision stages average 215.21 ms per round on MNIST and 791.03 ms on CIFAR-10. They account for 0.58% and 0.89% of the corresponding computation totals. STD-DAGMM adds auxiliary fitting, averaging 15.76 and 42.09 seconds. The checking subtotal groups these stages. The final rows give BaitFL's absolute saving in seconds per round together with the percentage reduction relative to STD-DAGMM. Stage times include sample standard deviations across the 100 rounds, capturing variation between ordinary training and periodic decisions.

Figure 6 shows average computation and elapsed round time over the complete horizon. BaitFL uses 29.35% less computation time on MNIST and 26.23% less on CIFAR-10 in these executions. Elapsed-time reductions are 34.87% and 27.20%. Research logs, checkpoints and other runtime costs contribute to the difference between computation and elapsed time. Network communication is absent from both measurements.

These timing cases describe different native workflows. BaitFL changes probing and participation, whereas STD-DAGMM keeps the full population. Removing free riders does not remove honest clients' local SGD jobs. Checking accounts for less than 1% of measured computation, while the STD-DAGMM comparison measures a particular fitting budget. Detailed per-round and fitting-stage records accompany the study.

The unassigned-runtime entry is the difference between elapsed time and the sum of explicitly timed computation and I/O stages. Its larger value for STD-DAGMM shows that the two execution paths include different untimed work. It is excluded from computation totals and cannot be attributed to detector fitting. Elapsed-time savings include this difference, so they are not an isolated measure of detection overhead.

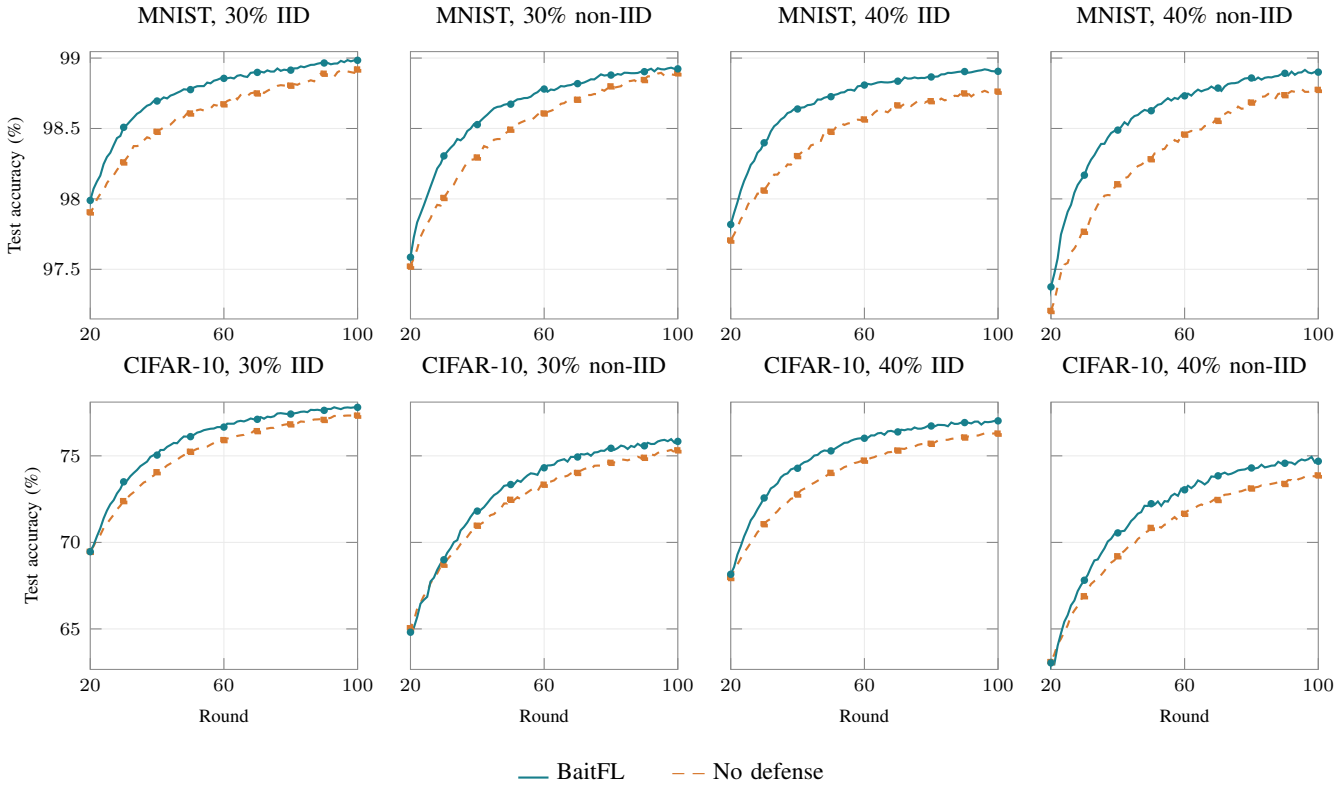


Fig. 5. Global test accuracy during training.

Low checking cost does not imply immediate detection. Figure 7 shows the variation in complete-removal rounds across configurations. Markers indicate the mean completion round, with whiskers spanning the observed minimum and maximum. The wider ranges in history-based attacks reflect the additional challenge cycles sometimes needed for confirmation.

VI. Complexity and Lightweight Design

A. Computational Cost

BaitFL reads model responses to extract small summaries, then compares those summaries when challenge coverage is complete. Let N be the largest active population, P the number of floating model coordinates, and L the number of floating tensors, including floating buffers. The detector uses B bootstrap replicates and $K = \lceil \sqrt{N} \rceil$ histogram bins. Table VIII separates response processing from the periodic decision work. Ordinary aggregation, local training and persistent research logs are outside these incremental bounds.

Response summaries require a pass over model coordinates. The more important population-scaling cost lies in the leave-one-out comparisons: each client is compared with the other $N-1$ profiles. This requires $O(N^2 L)$ coordinate work, with $O(N^2 \log N)$ additional work from repeated scalar medians in the current Python implementation. Bootstrap resampling acts on scalar scores rather than full model vectors.

For T rounds, M complete paired cycles and A sparse audits, a bound on total detector work is

$$W = O\big(TNP + (M+A)\big[N^2(L+\log N) + B(N+K)\big]\big). \tag{16}$$

The audit term includes single-challenge scoring, which cannot itself authorize removal. Dense monitoring increases response processing, but only the preselected pair enters a complete-cycle decision. Thus, the implementation is linear in model size for feature extraction and quadratic in client count for peer comparison.

At 100 clients, the two-challenge comparison covers 158,400 profile coordinates on MNIST and 435,600 on CIFAR-10, before constants for medians and distances. Each bootstrap view draws 100,000 scalar indices with the configured 1,000 replicates. These operations explain the small feature representation. Their practical cost is measured separately in Section V-H.

B. Memory and Communication

The detector's working state is $O(P + NL + BN + BK)$, including shared challenges, paired summaries and bootstrap temporaries. Evidence counters and fingerprints add $O(N)$. Table IX gives calculated payloads for the evaluated architectures. Two float32 challenges occupy about 1.58 MiB on MNIST and 4.74 MiB on CIFAR-10. A packed representation of the paired summaries requires about 8.59 and 19.53 KiB, respectively, at 100 clients. The calculation uses $2N(L+3)\cdot 4$ bytes, allowing three scalar slots alongside each profile.

TABLE VII
ROUND-TIME BREAKDOWN.

Stage	MNIST		CIFAR-10	
	BaitFL	STD-DAGMM	BaitFL	STD-DAGMM
Honest local training	34.30 ± 0.94	33.89 ± 0.66	81.37 ± 5.40	72.42 ± 3.68
Fabricated responses	0.018 ± 0.029	0.066 ± 0.011	0.188 ± 0.172	0.224 ± 0.022
Client diagnostic evaluation	0.161 ± 0.248	0	0.899 ± 0.752	0
Traps and scheduling	0.0018 ± 0.0055	0	0.018 ± 0.058	0
Response features and preparation	0.198 ± 0.050	0.045 ± 0.003	0.729 ± 0.179	0.115 ± 0.012
Auxiliary detector fitting	0	15.76 ± 1.25	0	42.09 ± 1.87
Scoring and state decisions	0.016 ± 0.071	0.335 ± 0.039	0.043 ± 0.145	0.827 ± 0.057
Aggregation	0.030 ± 0.013	0.043 ± 0.003	0.096 ± 0.039	0.136 ± 0.014
Global evaluation	2.20 ± 0.10	2.13 ± 0.04	5.21 ± 0.44	4.24 ± 0.20
Research logs and checkpoints	3.99 ± 0.61	3.75 ± 0.54	11.34 ± 1.79	9.76 ± 1.30
Unassigned runtime	0.0088 ± 0.0026	6.82 ± 2.33	0.054 ± 0.044	7.49 ± 2.08
Checking subtotal (included above)	0.215 ± 0.093	16.14 ± 1.25	0.791 ± 0.244	43.03 ± 1.91
Checking share of computation (%)	0.58	30.89	0.89	35.84
BaitFL time saved relative to STD-DAGMM				
Computation saved: s/round (%)	15.34 (29.35%)		31.50 (26.23%)	
Elapsed saved: s/round (%)	21.91 (34.87%)		37.35 (27.20%)	

Times are mean $\pm$ sample SD in seconds per round over 100 rounds. Combined stages are summed per round before calculating SD. Checking subtotals are included in the preceding stages. Shares and savings use complete-horizon means, relative to STD-DAGMM for savings.

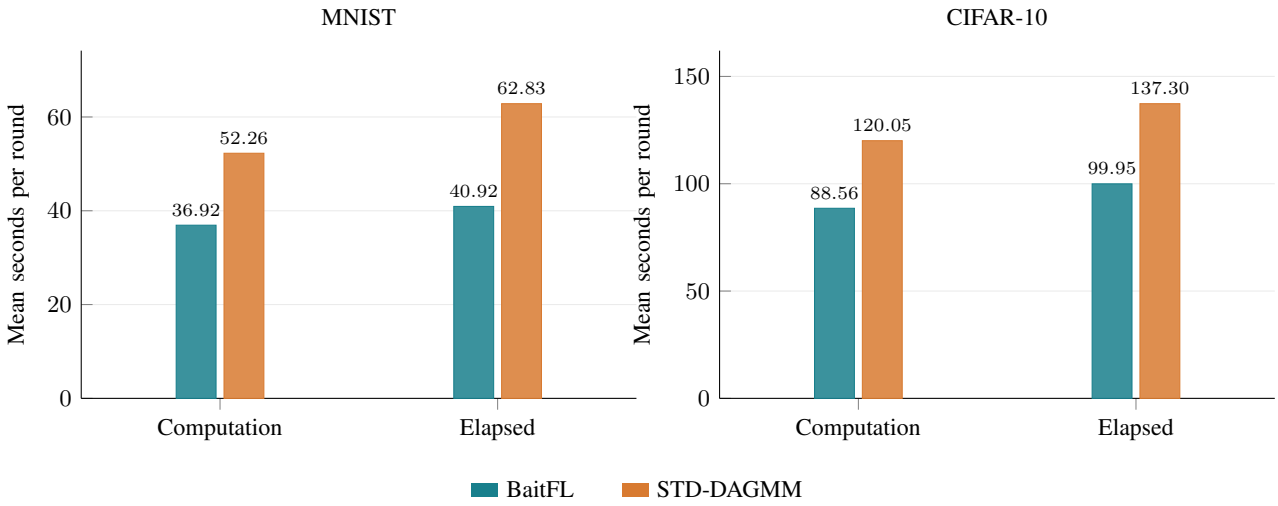


Fig. 6. Average computation and elapsed time per round.

These payloads describe packed numerical data, whereas the implementation also allocates Python objects, dictionaries and temporary arrays. Bootstrap gathers, indices and counts can coexist in memory. The full simulator additionally retains model responses, attacker histories and research logs. In dense probing, profile logs alone can grow as $O(TNL)$. Consequently, the process RSS in Table V measures the complete experiment rather than the detector's compact evidence state.

Challenges replace ordinary models in the existing request/response exchange. Clients follow the same local training procedure, and the nominal model-payload order remains $O(NP)$ per round. Challenge responses are withheld from aggregation, however, so an unchanged number of exchanges does not establish an unchanged time to target accuracy. Transport costs are not measured here.

The lightweight design rests on compact response features, a fixed number of evidence counters per client and the absence of auxiliary learning. The measured checking costs support that description at the evaluated scale. Larger populations still incur the quadratic peer comparison cost, and network or energy savings require separate measurements.

VII. DISCUSSION AND LIMITATIONS

A. Information Available to the Server

BaitFL uses returned models, client and challenge identifiers, response norms, profile distances and exact-sendback checks. Attacker identities and behavior settings belong to the simulator and evaluation records. They do not enter screening, geometric tests or confirmation. Likewise, the configured attacker proportion does not set a removal quota. This separation prevents direct use of evaluation labels, although the method's design has been informed by experimental observations.

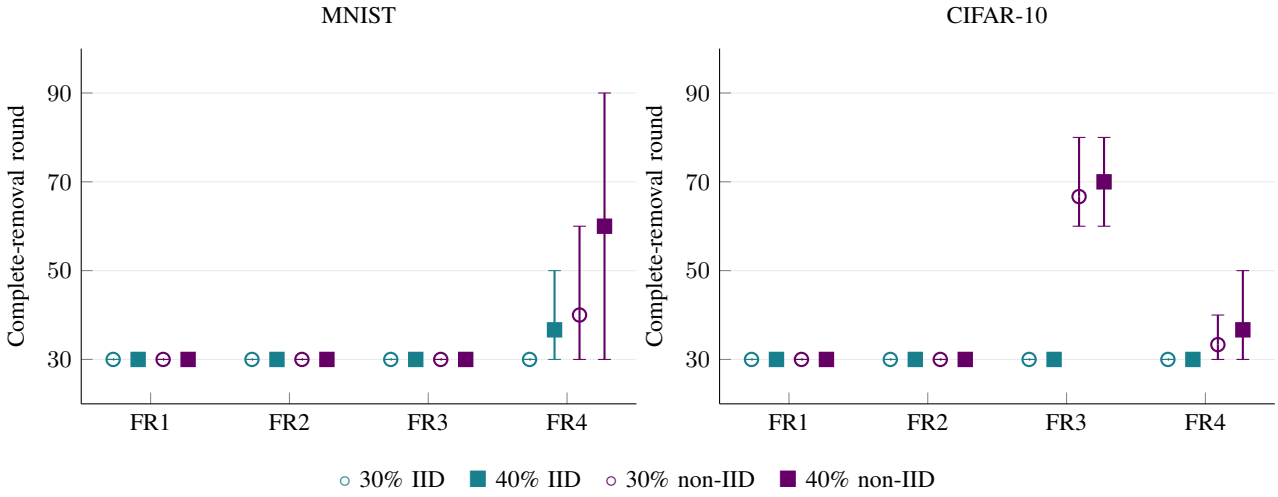


Fig. 7. Free-rider removal completion rounds.

TABLE VIII
INCREMENTAL DETECTOR COMPUTATION AND WORKING STATE.

Operation	Work	Working state	Implementation
Norms, profiles and fingerprints	$O(NP)$ per round	$O(P)$ scratch and $O(N)$ hashes	Read returned tensors without training an auxiliary model.
Selected challenge summaries	$O(NL)$ per paired cycle	$O(NL)$	Retain two small profiles per client.
Leave-one-out comparisons	$O(N^2(L + \log N))$ per cycle	$O(NL)$ temporary	Form peer medians and distances for each client.
Histogram and bootstrap splits	$O(B(N + K))$ per cycle	$O(BN + BK)$ temporary	Resample scalar scores across three views.
Geometric tests and confirmation	$O(N)$ per cycle	$O(N)$ counters	Apply fixed-component tests and update evidence memory.
Shared challenge construction	$O(P)$ per cycle	$O(P)$	Generate two shared models.

TABLE IX
CALCULATED DETECTOR PAYLOAD SIZES.

Model	Parameters	P	L	Two traps (MiB)	Pair payload (KiB)	Index array (MiB)	Counts (MiB)
MNIST	206922	206922	8	1.579	8.594	0.763	0.076
CIFAR-10	620810	621258	22	4.740	19.531	0.763	0.076

B. Confirmation and Honest-Client Safety

Statistical removal requires fresh evidence from the same component on both selected challenges and in another complete cycle. Mixed norm/profile findings cannot be combined to reach confirmation, and stored evidence alone cannot authorize removal. Paired exact sendback also requires repetition. Weak candidate labels do not renew quarantine, and unconfirmed review expires when fresh strong evidence does not recur.

These rules reduce reliance on isolated anomalies. They do not make repeated evidence statistically independent. An honest client keeps its local data across challenges and can therefore remain unusual in successive cycles. Certificate memory can accumulate such findings unless two complete clear cycles reset it, explaining the risk of late false removal observed in the experiments.

The decision boundaries are derived from the observed population rather than an attack-fitted numerical z-score cutoff. Nevertheless, the method has design parameters, including challenge strength, bootstrap confidence, cohort support and confirmation lengths. At 100 clients, statistical certification requires a cohort of at least ten. A small group of free riders that avoids exact sendback may therefore remain uncertified.

C. Adaptive Attacks and Identifiability

A response-only detector cannot distinguish two clients whose observable histories are identical, even if one trained and the other fabricated its responses. BaitFL therefore tests whether challenges expose differences in behavior. Its certificates do not prove that local computation occurred.

The evaluated free riders are not told which models are challenges. An adaptive participant might still recognize repeated frozen models, unusual perturbations or differences revealed through communication with other clients. Reproducible simulation randomness also does not establish unpredictability against an attacker who knows its configuration. Deployment would require server-secret randomness and tests against

challenge-aware strategies. Secrecy alone would not guarantee that challenge models are indistinguishable from ordinary ones.

D. Privacy and Participation Limits

The server inspects individual returned models, so the current protocol cannot be used directly with standard secure aggregation [13]. It does not access raw client data or require reported training loss, but these restrictions do not provide differential privacy or prevent information leakage through updates [2]–[4].

The reference population assumes an honest majority. Colluding clients could share trained responses, while Sybil identities could distort the reference distribution. Neither case is evaluated. The experiments also assume complete participation. Incomplete challenge coverage cannot certify removal, and the simulator stops on a nonfinite response rather than providing a full malicious-message or dropout-handling protocol. Checkpoint recovery restores the recorded training and evidence state after an interruption. Authentication and availability defenses require additional mechanisms.

The sparse-audit clock has an additional availability limitation. An audit whose findings are all blocked by cooldown can clear the clock without opening review, preventing subsequent sparse audits. This control-flow case was not observed in the completed study runs. Correcting it requires a versioned implementation change and a new evaluation, since the results here describe the implemented protocol.

E. Evaluation Scope and Future Work

The study evaluates three seeds per configuration, 100 clients, one architecture per dataset and one non-IID concentration. The pooled detection rates describe these experiments. They are not statistical guarantees for new populations. The evaluation does not include an independent held-out design study, and matched no-defense coverage is partial. Timing uses one declared non-IID FR3 configuration on each dataset. We make no comparative detection-accuracy claim against STD-DAGMM, PASS, FRAD or FRIDA, and the privacy comparison concerns information requirements rather than measured leakage.

Reducing honest-client errors is the first priority. Matched ablations should separate the effects of norm evidence, layer profiles, challenge strength and retained certificates. Honest-only populations and longer runs would help assess whether persistent data heterogeneity produces repeated false evidence. Broader tests should vary local training, non-IID concentration, attacker history and noise, and model architecture. For CIFAR-10, trainable-weight evidence should also be distinguished from the contribution of BatchNorm buffers. Honest-only cases and 5–20% free-rider populations would test the cohort gate's operating range. Published decaying-noise and scaled-delta strategies need separate tests [7], [10], rather than being treated as equivalent to FR1–FR4.

Adaptive free riders and changing client populations pose further questions. Experiments with collusion, intermittent participation and selective training should examine whether challenges remain informative when clients can infer the audit schedule. Privacy-preserving auditing also deserves investigation because the current method relies on individual response visibility. Benchmarks such as LEAF and FedScale offer natural next settings for client heterogeneity and system variation [30], [31].

Larger-scale profiling should measure detector memory, communication bytes, energy and time to a target model accuracy. Reusing peer computations and streaming research logs may reduce server cost, but those optimizations need their own implementation and evaluation. The present timing results establish costs for the recorded workflows and fitting budget, rather than a universal runtime advantage.

F. Code and Evidence Availability

The project repository is https://github.com/Aashnan-Rahman/Mitigating-Free-Rider. The study retains configurations, per-client decisions, per-round timing and source hashes for reproducibility. A public immutable archive of the exact manuscript evidence is not yet available.

VIII. Conclusion

BaitFL audits federated-learning responses through model challenges, compact statistical features and repeated fresh evidence. Multi-seed experiments on MNIST and CIFAR-10 remove all evaluated free riders, achieving 100.00% recall and 99.91% pooled precision. The 3 honest removals show the remaining difficulty of separating fabrication from persistent non-IID variation.

Available matched no-defense comparisons show mean final accuracy gains of 0.10 percentage points on MNIST and 0.60 on CIFAR-10. The separate instrumented runs place checking at 0.58% and 0.89% of computation time, respectively, without auxiliary detector training. Together, the results support lightweight response auditing at the evaluated scale. Improving honest-client protection while retaining detection under adaptive behavior is the next step.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[2] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021. [Online]. Available: https://arxiv.org/abs/1912.04977

[3] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/60a6c4002cc7b29142def8871531281a-Abstract.html

[4] L. Melis, C. Song, E. D. Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *IEEE Symposium on Security and Privacy*, 2019. [Online]. Available: https://arxiv.org/abs/1805.04049

[5] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 108. PMLR, 2020, pp. 2938–2948. [Online]. Available: https://proceedings.mlr.press/v108/bagdasaryan20a.html

[6] J. Lin, M. Du, and J. Liu, "Free-riders in federated learning: Attacks and defenses," arXiv preprint arXiv:1911.12560, 2019. [Online]. Available: https://arxiv.org/abs/1911.12560

[7] Y. Fraboni, R. Vidal, and M. Lorenzi, "Free-rider attacks on model aggregation in federated learning," in *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 130. PMLR, 2021, pp. 1846–1854. [Online]. Available: https://proceedings.mlr.press/v130/fraboni21a.html

[8] J. Wang, X. Chang, J. Mišić, V. B. Mišić, and Y. Wang, "PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2024. [Online]. Available: https://arxiv.org/abs/2207.07292

[9] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, "FRIDA: Free-rider detection using privacy attacks," *Journal of Information Security and Applications*, vol. 97, 2026, art. no. 104357. [Online]. Available: https://arxiv.org/abs/2410.05020

[10] Z. Zhu, J. Shu, X. Zou, and X. Jia, "Advanced free-rider attacks in federated learning," in *NeurIPS Workshop on New Frontiers in Federated Learning*, 2021. [Online]. Available: https://neurips2021workshopfl.github.io/NFFL-2021/papers/2021/Zhu2021.pdf

[11] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=BJJLHbb0-

[12] B. Wang, H. Li, X. Liu, and Y. Guo, "FRAD: Free-rider attacks detection mechanism for federated learning in AIoT," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024. [Online]. Available: https://doi.org/10.1109/JIOT.2023.3298606

[13] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191. [Online]. Available: https://research.google/pubs/practical-secure-aggregation-for-privacy-preserving-machine-learning/

[14] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/6617-machine-learning-with-adversaries-byzantine-tolerant-gradient-descent.pdf

[15] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 5650–5659. [Online]. Available: https://proceedings.mlr.press/v80/yin18a.html

[16] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Transactions on Signal Processing*, vol. 70, pp. 1142–1154, 2022. [Online]. Available: https://arxiv.org/abs/1912.13445

[17] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Network and Distributed System Security Symposium*, 2021. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/ndss2021_6C-2_24434_paper.pdf

[18] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems*, vol. 2, 2020. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html

[19] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 5132–5143. [Online]. Available: https://proceedings.mlr.press/v119/karimireddy20a.html

[20] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: https://papers.neurips.cc/paper_files/paper/2020/hash/564127c03caab942e503ee6f810f54fd-Abstract.html

[21] H. Jia, M. Yaghini, C. A. Choquette-Choo, N. Dullerud, A. Thudi, V. Chandrasekaran, and N. Papernot, "Proof-of-learning: Definitions and practice," in *IEEE Symposium on Security and Privacy*, 2021, pp. 1039–1056. [Online]. Available: https://arxiv.org/abs/2103.05633

[22] L. Zhao, Q. Wang, C. Wang, Q. Li, C. Shen, X. Lin, S. Hu, and M. Du, "VeriML: Enabling integrity assurances and fair payments for machine learning as a service," arXiv preprint arXiv:1909.06961, 2019. [Online]. Available: https://arxiv.org/abs/1909.06961

[23] A. Ghorbani and J. Zou, "Data shapley: Equitable valuation of data for machine learning," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 2242–2251. [Online]. Available: https://proceedings.mlr.press/v97/ghorbani19c.html

[24] R. Jia, D. Dao, B. Wang, F. A. Hubis, N. Hynes, N. M. Gurel, B. Li, C. Zhang, D. Song, and C. Spanos, "Towards efficient data valuation based on the shapley value," in *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 89, 2019. [Online]. Available: https://proceedings.mlr.press/v89/jia19a.html

[25] Z. Liu, Y. Chen, H. Yu, Y. Liu, and L. Cui, "GTG-Shapley: Efficient and accurate participant contribution evaluation in federated learning," *ACM Transactions on Intelligent Systems and Technology*, vol. 13, no. 4, 2022, art. no. 60. [Online]. Available: https://arxiv.org/abs/2109.02053

[26] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979. [Online]. Available: https://doi.org/10.1109/TSMC.1979.4310076

[27] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: https://doi.org/10.1214/aos/1176344552

[28] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://doi.org/10.1109/5.726791

[29] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html

[30] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečný, H. B. McMahan, V. Smith, and A. Talwalkar, "LEAF: A benchmark for federated settings," arXiv preprint arXiv:1812.01097, 2018. [Online]. Available: https://arxiv.org/abs/1812.01097

[31] F. Lai, Y. Dai, S. Singapuram, J. Liu, X. Zhu, H. Madhyastha, and M. Chowdhury, "FedScale: Benchmarking model and system performance of federated learning at scale," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 11 814–11 827. [Online]. Available: https://proceedings.mlr.press/v162/lai22a.html